Farmer WhatsApp Advisory Bot: Project Notes

Written September 2026, at the ideation stage. Nothing has been built yet. This is the "where my head was at" file, so future-me can pick this up without starting over.

---

1. The idea in one paragraph

A farmer texts a WhatsApp number with a problem, in plain language, in Hindi, Telugu, Tamil, or English. They can type, send a photo of a sick plant, or send a voice note. The bot replies with practical, sourced advice that takes into account where they are, the weather forecast, and current market prices. It should feel like asking a knowledgeable neighbour, not searching a website.

Why I care about it: it solves a real problem (farmers often can't reach an expert quickly), and it lets me build real ML and systems skills instead of a toy project.

---

2. Honest verdict on the idea (so I don't fool myself later)

- **The basic version is a weekend demo, not a resume project.** WhatsApp → n8n → Claude → reply is a wrapper. Anyone can copy it in an afternoon. Grade: about C+.
- **The space is crowded.** Kisan Call Centre, Kisan e-Mitra, Digital Green's Farmer.Chat, KissanAI, Plantix. "A chatbot for farmers" alone won't stand out.
- **It becomes strong through depth, not features.** The things that make it good are: models I trained myself, an evaluation with real numbers, safety thinking, and real users. Done that way, it's a B+ to A-.
- **"Wrapper" doesn't mean I did no work.** It means nothing in it is unique to me. My job is to add the parts only I could have made: my own test set, my own decisions, my own failure analysis.

---

3. What the bot should do

- Understand text, photos, and voice notes
- Reply in the farmer's language
- Use their location (WhatsApp lets you share a pin) for weather, soil, and nearby market prices
- Diagnose plant problems from photos, and say so honestly when it's unsure ("send me a close-up of the underside of the leaf")
- Answer with sources, e.g. "Source: ICAR advisory"
- Give weather-aware advice ("don't spray tomorrow, rain expected")
- Tell them today's mandi price for their crop
- **Never** state pesticide dosages unless a retrieved source says so. Wrong advice can destroy a crop.

Known limit: soil can't be reliably judged from a photo. Soil info comes from location-based datasets (SoilGrids etc.), and I should say that openly in the project.

---

4. The layers (the whole system, top to bottom)

#	Layer	What it does
1	WhatsApp connection	Receives messages, sends replies
2	Speech-to-text	Turns voice notes into text
3	Crop disease from photos	Looks at a leaf and says what's wrong
4	Knowledge retrieval (RAG)	Finds relevant passages from trusted agriculture documents
5	Language model	Writes the final reply in the farmer's language
6	Agent logic	Decides what to look up (weather, prices, documents)
7	Data and tools	Weather, mandi prices, soil data
8	Evaluation	Measures whether any of it actually works

---

5. Build vs. borrow: where I landed

I went back and forth on how much to build myself. The honest conclusion is that nobody builds everything from scratch, and I shouldn't try. What matters is that **the hard, interesting parts are mine, and only the commodity parts are borrowed.**

My current plan: I build layers 3 and 4. I borrow the rest.

Layer	Decision	Reason
1. WhatsApp	My code, on top of Meta's API	Plumbing, but it's my server
2. Speech	Borrow (Sarvam / Whisper API)	Fine-tuning for Telugu/Tamil is hard mostly because of data. Stretch goal only.
3. Photo diagnosis	**Build (train my own classifier)**	Real ML, the most "mine" piece
4. Retrieval (RAG)	**Build (own chunking, embeddings, search)**	Free, works in all three languages, makes answers grounded
5. Language model	Borrow (Claude)	Self-hosting is too costly and much worse in Indian languages
6. Agent logic	Mine (simple loop, no heavy framework)	Easy to write, easy to explain
7. Data tools	Borrow free APIs	Open-Meteo, Agmarknet, SoilGrids
8. Evaluation	**Mine, always**	This is what makes the project credible

Why not layer 5 (own language model)? Small open models are noticeably worse in Hindi/Telugu/Tamil, and keeping a GPU online for a live bot costs way more than my budget. Optional experiment later: fine-tune a small model with LoRA on agriculture Q&A and compare it against Claude. That's a research side-story, not the live bot.

Extra stretch: fine-tune Whisper-small for Hindi speech and report the word error rate before and after. Fine if it loses to Sarvam. "I measured it and here's why" is still a good result.

---

6. Build order (always keep something working)

1. **Layer 1 first.** A plain echo bot, then plug in Claude. Now there's a live demo.
2. **Layer 4 (RAG).** Text questions get cited, grounded answers.
3. **Layer 3 (image classifier).** Photos work, and I get a benchmark against Claude's vision.
4. **Speech.** Borrowed API first. Only replace it with my own model if I have time.
5. **Evaluation and real users.** Run through all of it, write it up.

Each step swaps a borrowed piece for my own, so I always have something to demo, and each swap produces a comparison I can report.

---

7. Notes on the parts I'm building

Photo classifier (layer 3)

- **Data:** PlantVillage (clean lab photos, easy) and PlantDoc (real field photos, much harder).
- **Model:** fine-tune a small pretrained model (EfficientNet or MobileNet). This is transfer learning, standard practice.
- **The interesting finding will be the gap:** expect 95%+ on PlantVillage but much lower on messy real photos. That mismatch is the story.
- **Add an "I'm not sure" threshold.** A classifier that knows when it doesn't know is safer for farmers.
- **Compare against Claude vision** on the same test images.
- **Compute:** free on Kaggle (about 30 GPU hours a week) or Google Colab.

Retrieval (layer 4)

- **Sources:** ICAR advisories, state agricultural university portals (e.g. TNAU for Tamil Nadu, PJTSAU for Telangana), government crop guides.
- **Write my own:** document loading, chunking, embedding, search, reranking.
- **Embeddings:** an open multilingual model (multilingual-e5 or BGE-M3) that handles Hindi/Telugu/Tamil.
- **Search:** FAISS or plain numpy. No hosted vector database.
- **Experiments to run:** fixed-size chunks vs. section-aware chunks, with and without a reranker.
- **How I judge it:** does the right passage come back in the top 3 results?

---

8. What makes this *mine* (the part that matters most)

The plumbing can be AI-generated. These I do or at least decide myself:

1. **My own test set.** 100 to 200 real questions and photos that I collect, label, and check against agronomy sources. Nobody else has this.
2. **Domain decisions.** Which crops, which region, which sources I trust, when the bot should refuse. Talking to even 3 to 5 farmers or an extension worker will teach me things no tutorial covers.
3. **Failure analysis.** Find where it breaks (e.g. "confuses early and late blight", "noisy Telugu voice notes lose accuracy"), fix it, measure the improvement.
4. **The data work.** Cleaning documents, picking chunk sizes, handling missing mandi data.
5. **A small pilot.** Real people, real messages, and what I changed because of it.

My rule for building with AI: let AI write the boilerplate, but I own the eval, the safety rules, the "why did we choose X" decisions, and reading the failure cases.

How I'll make sure I can explain it:

- Build one module at a time, and read and explain it in my own words before moving on.
- Keep a `DECISIONS.md` that records why I chose each API, model, or structure.
- Be able to draw the architecture from memory.

---

9. Practical gotchas to remember

- **Twilio's WhatsApp sandbox isn't a public number.** Users must send a join code first. A real public number needs a Meta-approved WhatsApp Business sender, which means business verification.
- **Better: Meta's WhatsApp Cloud API directly.** It gives a free test number and a monthly allowance of free service conversations. Public launch still needs a verified business account.
- **Voice notes and photos arrive as URLs** that must be downloaded with authentication.
- **Meta retries messages,** so I need deduplication.
- **Mandi price data is patchy and sometimes delayed.** Handle missing data gracefully and always say which mandi and what date.
- **Telugu and Tamil speech recognition is weaker than Hindi.** Measure accuracy per language and report it honestly.

---

10. Budget (about ₹200, roughly $2.40)

Item	Cost
WhatsApp Cloud API (test number)	₹0
Weather, prices, soil APIs	₹0
Hosting (Render/Railway free tier, or local + ngrok)	₹0
Model training (Kaggle/Colab)	₹0
Embeddings, FAISS	₹0
Claude API (use Haiku 4.5 by default, cache the system prompt)	about ₹50 to ₹100
Speech-to-text	₹0 to ₹50 (check Sarvam's current free credits)

The real budget is tokens. At about ₹0.5 per query I get roughly 400 test queries. That's enough for building and evaluating.

---

11. Time and payoff

Tier	Time	What I have	Resume value
Demo	1 weekend	Text bot with location, weather, prices	B-. Forgettable.
Solid	2 to 3 weekends	Plus photos, RAG with citations, Hindi voice	B+. Real system.
Standout	4 to 6 weekends	Plus my own classifier, my own RAG, an eval report, a small pilot, Telugu/Tamil	A-. Memorable.

The jump from B+ to A- comes from the eval and the pilot, not from extra features. If I'm short on time, cut features and keep the eval.

---

12. Stack

Python + FastAPI, Meta WhatsApp Cloud API, SQLite for user state (location, crop, recent messages), FAISS for vectors, PyTorch for the classifier. Skip n8n except for a throwaway prototype.

---

13. What I'd like the resume line to say

> Built a multilingual WhatsApp advisory agent for farmers. Trained and evaluated a crop-disease image classifier and benchmarked it against a frontier vision model. Built a retrieval pipeline over ICAR advisories with source citations. Tested across N questions and photos in Hindi, Telugu, and Tamil, with a pilot with real farmers.

The numbers in that line only exist if I run the eval. That's why the eval isn't optional.

---

14. Open questions to decide when I restart

- Which state and crops first? (Picking one region makes the data and documents much easier.)
- Can I find 3 to 5 real farmers or an extension worker to talk to?
- Hindi only first, or all three languages from the start? (I lean toward Hindi first, then add.)
- Sarvam or Bhashini for speech? Check current pricing and free credits.
- Do I do the LoRA language-model experiment at the end, or skip it?

---

15. Next step when I'm back

1. Pick the region and crops.
2. Set up the repo with a `DECISIONS.md` and a folder for each layer.
3. Get the WhatsApp echo bot running on Meta's test number.
4. Start collecting the eval questions early. It takes longer than you think.
5. Start the classifier training on Kaggle while building everything else around it.